Figure 1.4 A User API with two consumers and two independent contracts

When the mobile application makes a `GET` request to the `/user/1` endpoint, they will see the additional field `favorites`, but this shouldn't introduce any unexpected problems to the web application because the contract defined with the User API still stands true. From the User API perspective, they are still meeting the requirements of both the web and mobile applications and sending the same set of responses, but only the mobile application is using the additional `favorites` field.

The contracts generated by web and mobile application consumers are also uploaded to a shared broker, meaning multiple teams can see the different contracts easily. It also makes it easier for the team responsible for the User API to track who their consumers are. Having a shared broker for all the contracts makes it easier to visualize which teams have dependencies among each other so that any contract changes can be spotted more easily.

Another reason contract testing matters is that the cost of a bug gets exponentially more expensive when it's not found early, which Rafaela Azevedo explained in detail in her blog post (https://mng.bz/Dpa0). Contract testing combines the best qualities of an API integration test and a unit test. It combines the high confidence that an API integration test provides with the speed, low maintenance, and cheaper cost of running a unit test. Contract testing can also help minimize the number of integration or end-to-end (E2E) tests, which are also proven to be slow, flaky, and prone to errors.

> **TIP** If your team already has a substantial number of integration tests, many of these tests can also be shifted earlier in the development process to the contract level. However, this doesn't entirely replace the need for integration or E2E testing because contract testing doesn't verify any environmental problems caused by configuration problems or unexpected behaviors caused by system side effects. We cover this in more detail as part of chapter 2.

Apart from detecting bugs missed by proper communication among teams, there are also other reasons why contract testing matters (not an exhaustive list):